

Процессор – это контейнер
 Поток – это исполнитель внутри контейнера

GMP Модель

Идея: горутины распределяются по потокам ОС

Какие есть подходы?

- N:1 – N горутин привязываются к одному потоку
- 1:1 – одна горутина привязывается к одному потоку
- M:N – M горутин привязываются к N потокам, что позволяет полностью использовать несколько ядер ЦП для эффективной работы процессов программы.

Буква	Что это	Кто создает
G	Goroutine – легковесный поток, <code>go func()</code>	Разработчик, через код
M	Machine – системный поток ОС (OS thread)	Планировщик Go
P	Processor – логический процессор, содержит локальную очередь готовых G	Планировщик Go (кол-во = GOMAXPROCS)

НО логический процессор P планировщика Go $\neq$ ядро ЦП

С версии Go 1.5 $GOMAXPROCS = L$

$$L = P \cdot T_p$$

L – кол-во лог. процессоров CPU

P – число физических ядер ЦП

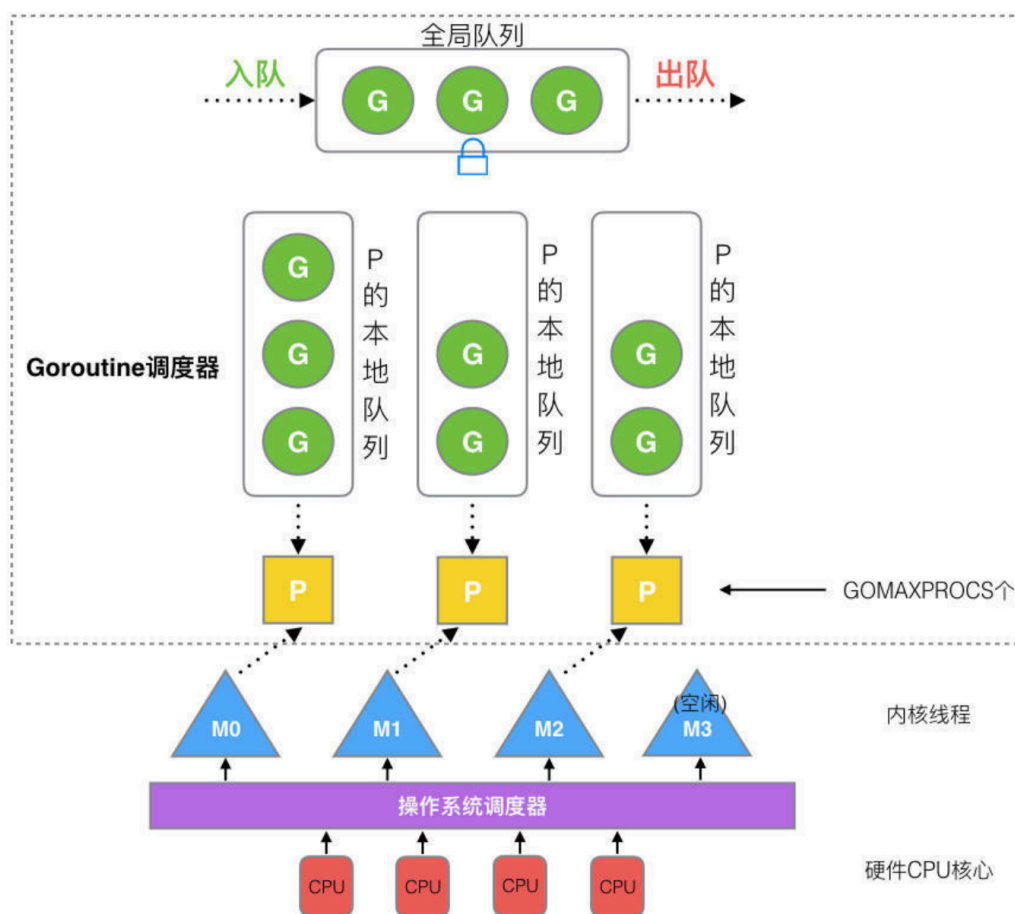
T_p – кол-во потоков, поддерживаемых 1 ядром

от чего зависит T_p :

- Без технологии многопоточности: 1
- С технологией Hyper-Threading Intel / SMT AMD: 2
- Гибридные процессоры:
 - мощные ядра: 2
 - простые: 1

Но можно вручную сделать так: `runtime.GOMAXPROCS(1)`

ОДНА ГОРУТИНА G НА ОДИН M В КАЖДЫЙ МОМЕНТ ВРЕМЕНИ



Каждый P имеет свою собственную очередь хранения горутин на исполнение, и также есть глобальная очередь.

Когда в локальной очереди P горутин нет, он “крадет” сколько-то горутин из глобальной очереди. А если и глобальная очередь пуста, то он возьмет горутину из других локальных очередей по принципу выше.

При извлечении G (горутины) из глобальной очереди не следует забирать их в большом количестве за раз; в противном случае другие P (процессоры) будут приходить сюда, чтобы забрать их, что создаст избыточные накладные расходы.

Как это работает:

1. Системный поток (M), который хочет запустить горутину, сперва должен закрепиться за P
2. M берет горутину G из локальной очереди P на исполнение, а затем берет следующую горутину G
3. Если M встречает системный вызов (например файловое чтение/запись) пока исполняет G, M открепится от P (чтобы проц не простаивал), но M запомнит P. Когда G и M выйдут из системного вызова, они найдут этот P. Но если не получится найти P, то G будет помечена как готовая к выполнению и помещена в глобальную очередь

Механизмы решения Writer Starvation

```
type p struct {  
    // ...  
    runqhead uint32      // голова кольцевого буфера  
    runqtail uint32      // хвост кольцевого буфера  
    runq      [256]guintptr // кольцевой буфер на 256 горутин  
    runnext  uintptr      // следующая горутина (слот с высоким приоритетом!)  
    // ...  
}
```

1. Вытеснение (Preemption) – Долгоиграющий таймер

Как это работает:

Специальный системный поток `sysmon` просыпается примерно каждые 10 миллисекунд. Он смотрит, не заняла ли одна горутина процессор (P) слишком надолго. Если да, он посылает этой горутине сигнал, чтобы она “притормозила”. (после вытеснения горутина попадает в конец локальной очереди своего P)

В чем суть:

Этот механизм превращает кооперативный (добровольный) планировщик в практически вытесняющий. С версии Go 1.14, планировщик может прервать даже вечный цикл `for {}` чтобы дать шанс другим горутинам

2. Приоритет `runnext` – “Схватка за процессор”

Этот механизм борется с делегацией управления между горутинами, которое не позволяет никому другому выполниться

Как это работает:

У каждого процессора (P) есть слот `runnext` для одной горутин, которая будет выполнена следующей. Это как “быстрый трек” для одной горутин

В чем суть:

Если горутина A пробуждает горутину B, то B часто попадает в `runnext` и выполняется почти сразу. Проблема в том, что A и B могут бесконечно так делать друг с другом, используя `runnext`, и игнорировать остальные горутин в очереди.

Счетчик `p.schedtick` помогает этого избежать. Если горутина из `runnext` отработала слишком много “квантов” подряд, планировщик воспринимает это как признак потенциального голодания и дает другим шанс

3. Глобальная очередь и `schedtick` — “План Б”

Когда локальные очереди процессоров переполняются или работают несправедливо, в игру вступает глобальная очередь.

Как это работает:

Планировщик периодически, раз в 61 “тик” (`schedtick`), обязательно проверяет глобальную очередь на наличие ожидающих горутин .

В чем суть:

Это гарантирует, что даже если на каком-то процессоре (P) происходит “несправедливая” борьба за `runnext`, горутин из глобальной очереди все равно получают свой шанс на выполнение. Это как принудительный “сброс” для обеспечения общей справедливости.